

05_tarea_grafica_radial_pokemon

April 27, 2026

1 Tarea paso a paso: Gráfico Radial (Radar Chart)

¡ATENCIÓN! Lee esto antes de empezar: Esta tarea está diseñada paso a paso para que afiances los conceptos vistos en clase sobre los **Radar Charts**. Su objetivo es que practiques cómo comparar visualmente múltiples variables numéricas entre distintas categorías de manera clara y efectiva.

Nota importante sobre la Inteligencia Artificial: Es tentador copiar y pegar el enunciado de estos ejercicios en ChatGPT o Claude para que te den la solución... **Hacer eso perjudicará gravemente tu aprendizaje. (CODEA UN POCO QUE TE ENTERES QUE HACES).** La única forma de asimilar la sintaxis de las herramientas de visualización es escribiéndolas. **Usa tu cerebro, actúa paso a paso, equivócate, lee la documentación de Plotly, busca tu fallo y ¡aprende de él!**

1.1 Preparación y Datos

Para esta tarea vamos a utilizar el famoso dataset **“Pokémon”**, un conjunto de datos que recoge las estadísticas de combate de todos los Pokémon de las primeras 6 generaciones. Es un dataset ideal para un **Radar Chart** porque cuenta con 6 variables numéricas de combate (HP, Attack, Defense, Sp. Atk, Sp. Def, Speed) y una columna categórica (Type 1) que nos permite comparar tipos entre sí.

1. Descarga el dataset de Kaggle: [Pokémon - Kaggle](#)
2. Guarda el archivo `Pokemon.csv` en la carpeta `data/` de tu proyecto.
3. En la siguiente celda, importa las librerías necesarias (`pandas`, `matplotlib.pyplot`, `plotly.graph_objects`) y carga el CSV en un DataFrame llamado `df`.

```
[1]: import pandas as pd
import matplotlib.pyplot as plt
import plotly.graph_objects as go
from math import pi

# CARGA DE DATOS
df = pd.read_csv("data/Pokemon.csv")

df.head(5)
```

```
[1]:      #      Name Type 1  Type 2  Total  HP  Attack  Defense  \
0  1      Bulbasaur  Grass  Poison    318  45     49     49
```

1	2	Ivysaur	Grass	Poison	405	60	62	63
2	3	Venusaur	Grass	Poison	525	80	82	83
3	3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123
4	4	Charmander	Fire	NaN	309	39	52	43

	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	65	65	45	1	False
1	80	80	60	1	False
2	100	100	80	1	False
3	122	120	80	1	False
4	60	50	65	1	False

```
[2]: # Ver la información del dataset
df.describe(include="all")
```

```
[2]:
```

	#	Name	Type 1	Type 2	Total	HP \
count	800.000000	800	800	414	800.000000	800.000000
unique	NaN	800	18	18	NaN	NaN
top	NaN	Bulbasaur	Water	Flying	NaN	NaN
freq	NaN	1	112	97	NaN	NaN
mean	362.813750	NaN	NaN	NaN	435.10250	69.258750
std	208.343798	NaN	NaN	NaN	119.96304	25.534669
min	1.000000	NaN	NaN	NaN	180.00000	1.000000
25%	184.750000	NaN	NaN	NaN	330.00000	50.000000
50%	364.500000	NaN	NaN	NaN	450.00000	65.000000
75%	539.250000	NaN	NaN	NaN	515.00000	80.000000
max	721.000000	NaN	NaN	NaN	780.00000	255.000000

	Attack	Defense	Sp. Atk	Sp. Def	Speed \
count	800.000000	800.000000	800.000000	800.000000	800.000000
unique	NaN	NaN	NaN	NaN	NaN
top	NaN	NaN	NaN	NaN	NaN
freq	NaN	NaN	NaN	NaN	NaN
mean	79.001250	73.842500	72.820000	71.902500	68.277500
std	32.457366	31.183501	32.722294	27.828916	29.060474
min	5.000000	5.000000	10.000000	20.000000	5.000000
25%	55.000000	50.000000	49.750000	50.000000	45.000000
50%	75.000000	70.000000	65.000000	70.000000	65.000000
75%	100.000000	90.000000	95.000000	90.000000	90.000000
max	190.000000	230.000000	194.000000	230.000000	180.000000

	Generation	Legendary
count	800.000000	800
unique	NaN	2
top	NaN	False
freq	NaN	735
mean	3.32375	NaN

std	1.66129	NaN
min	1.00000	NaN
25%	2.00000	NaN
50%	3.00000	NaN
75%	5.00000	NaN
max	6.00000	NaN

```
[3]: # Limpieza de nulos
df = df.dropna()
df
```

```
[3]:
```

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	\
0	1	Bulbasaur	Grass	Poison	318	45	49	49	
1	2	Ivysaur	Grass	Poison	405	60	62	63	
2	3	Venusaur	Grass	Poison	525	80	82	83	
3	3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123	
6	6	Charizard	Fire	Flying	534	78	84	78	
..	...	...	...	...	...	...	...	...	
795	719	Diancie	Rock	Fairy	600	50	100	150	
796	719	DiancieMega Diancie	Rock	Fairy	700	50	160	110	
797	720	HoopahHoop Confined	Psychic	Ghost	600	80	110	60	
798	720	HoopahHoop Unbound	Psychic	Dark	680	80	160	60	
799	721	Volcanion	Fire	Water	600	80	110	120	

	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	65	65	45	1	False
1	80	80	60	1	False
2	100	100	80	1	False
3	122	120	80	1	False
6	109	85	100	1	False
..	...	...	...	...	...
795	100	150	50	6	True
796	160	110	110	6	True
797	150	130	70	6	True
798	170	130	80	6	True
799	130	90	70	6	True

[414 rows x 13 columns]

1.2 Paso 1: Filtrar y Agrupar los Datos

Objetivo: Preparar el DataFrame para el gráfico radial. Necesitamos resumir las estadísticas de combate para cada tipo de Pokémon calculando la **media** de cada variable numérica.

Instrucciones: 1. Crea una lista llamada **stats** con los nombres de las 6 columnas numéricas de combate: 'HP', 'Attack', 'Defense', 'Sp. Atk', 'Sp. Def' y 'Speed'. 2. Usando **df**, selecciona únicamente las columnas de **stats** más la columna 'Type 1', agrupa por 'Type 1' y calcula la media (**.mean()**). Guarda el resultado en una variable llamada **agrupado**. 3. Muestra **agrupado**

para ver el resultado.

```
[4]: # Las 6 stats de combate
stats = ['HP', 'Attack', 'Defense', 'Sp. Atk', 'Sp. Def', 'Speed']

# Por Type 1, media de cada stat (uso media porque quiero el perfil "tipico"
# del tipo)
agrupado = df[stats + ['Type 1']].groupby('Type 1').mean()
agrupado
```

```
[4]:
```

	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed
Type 1						
Bug	58.134615	77.711538	75.730769	58.634615	71.711538	66.173077
Dark	69.809524	88.190476	74.523810	71.476190	73.238095	73.047619
Dragon	94.571429	122.571429	96.476190	116.238095	97.857143	94.666667
Electric	59.470588	68.882353	83.764706	108.882353	85.352941	84.470588
Fairy	70.000000	45.000000	90.000000	100.000000	110.000000	60.000000
Fighting	66.142857	95.857143	75.142857	82.571429	69.857143	88.285714
Fire	78.291667	92.166667	78.708333	105.458333	80.166667	76.791667
Flying	62.500000	50.000000	57.500000	71.000000	60.000000	89.000000
Ghost	71.318182	72.545455	78.090909	82.045455	70.590909	69.500000
Grass	69.837838	77.135135	71.135135	81.243243	75.351351	62.108108
Ground	80.000000	101.315789	86.263158	62.421053	67.578947	68.842105
Ice	76.818182	69.090909	61.363636	75.000000	69.090909	67.727273
Normal	70.972973	72.972973	57.540541	56.432432	57.756757	75.000000
Poison	68.692308	77.461538	70.615385	63.923077	67.307692	74.076923
Psychic	66.789474	84.473684	68.578947	98.105263	94.052632	86.736842
Rock	64.914286	90.171429	99.142857	69.200000	79.885714	61.828571
Steel	65.954545	93.318182	125.090909	68.318182	78.045455	55.545455
Water	77.867925	76.735849	77.113208	76.886792	76.622642	63.830189

1.3 Paso 2: Radar Chart con Datos Brutos

Objetivo: Construir tu primer gráfico radial con Plotly usando los datos sin normalizar, y observar qué problema aparece.

Instrucciones: 1. Selecciona 3 tipos de Pokémon para comparar. Por ejemplo: 'Fire', 'Water' y 'Grass' (el trío clásico). Guárdalos en una lista `tipos_seleccionados`. 2. Extrae las estadísticas de cada tipo como una lista de Python usando `list(agrupado.loc['Fire', :])`, y haz lo mismo para los otros dos tipos. 3. Crea un `go.Figure()` y añade un `go.Scatterpolar` por cada tipo con los parámetros `r` (los valores), `theta` (la lista `stats`), `fill='toself'` y `name` (el nombre del tipo). 4. Actualiza el layout con `fig.update_layout(...)` para añadir un título y mostrar la leyenda. 5. Muestra el gráfico con `fig.show()`. ¿Qué problema observas con las escalas?

```
[5]: # Trío clásico Fire/Water/Grass (los iniciales del juego, tienen perfiles muy
# distintos)
tipos_seleccionados = ['Fire', 'Water', 'Grass']
```

```

# Listas de cada tipo (igual que adelie/chinstrap/gentoo en la teoría)
fire = list(agrupado.loc['Fire', :])
water = list(agrupado.loc['Water', :])
grass = list(agrupado.loc['Grass', :])
print('fire ', fire)
print('water ', water)
print('grass ', grass)

# Radar Chart con Plotly
# https://plotly.com/python/radar-chart/
fig = go.Figure()
fig.add_trace(go.Scatterpolar(
    r=fire,
    theta=stats,
    fill='toself',
    name='Fire'
))

fig.add_trace(go.Scatterpolar(
    r=water,
    theta=stats,
    fill='toself',
    name='Water'
))

fig.add_trace(go.Scatterpolar(
    r=grass,
    theta=stats,
    fill='toself',
    name='Grass'
))

# Aún sin tocar el eje radial: quiero ver que arranca en ~50 en vez de en 0
fig.update_layout(
    title='Fire vs Water vs Grass (datos brutos)',
    showlegend=True
)

fig.show()

```

```

fire [78.29166666666667, 92.16666666666667, 78.70833333333333,
105.45833333333333, 80.16666666666667, 76.79166666666667]
water [77.86792452830188, 76.73584905660377, 77.11320754716981,
76.88679245283019, 76.62264150943396, 63.83018867924528]
grass [69.83783783783784, 77.13513513513513, 71.13513513513513,
81.24324324324324, 75.35135135135135, 62.108108108108105]

```

Fire vs Water vs Grass (datos brutos)



1.3.1 Interpretación del Gráfico (Datos Brutos)

Como podemos observar, aunque el gráfico ya tiene forma de radar, **el eje radial empieza en un valor muy alto** (alrededor de 50-60 en lugar de 0). Esto hace que las diferencias entre tipos parezcan más pequeñas de lo que realmente son. Además, si las variables tuviesen escalas muy distintas (por ejemplo, una en decenas y otra en miles), el gráfico sería directamente inútil.

Solución: normalizar los datos para que todos los valores estén entre 0 y 1.

1.4 Paso 3: Normalización de los Datos

Objetivo: Escalar todas las estadísticas al rango $[0, 1]$ para que el gráfico radial sea comparativo y justo.

Instrucciones: 1. Selecciona del DataFrame `df` solo las columnas de `stats` y guárdalas en `df2`. 2. Aplica la fórmula de normalización Min-Max: $(df2 - df2.min()) / (df2.max() - df2.min())$. Guarda el resultado en `df_norm`. 3. Añade la columna 'Type 1' de vuelta al DataFrame normalizado: `df_norm['Type 1'] = df['Type 1']`. 4. Agrupa `df_norm` por 'Type 1' y calcula la media. Guarda en `agrupado_norm`. 5. Muestra `agrupado_norm` y verifica que todos los valores están entre 0 y 1.

```
[6]: # Podemos intentar NORMALIZAR nuestro dataset...
# Uso Min-Max porque para un radar necesito rango fijo y acotado, no Z-score
df2 = df[stats]
df_norm = (df2 - df2.min()) / (df2.max() - df2.min())
df_norm['Type 1'] = df['Type 1']

agrupado_norm = df_norm.groupby('Type 1').mean()
agrupado_norm
```

```
[6]:
```

	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed
Type 1						
Bug	0.383454	0.376175	0.282469	0.286086	0.246245	0.394665
Dark	0.461809	0.434392	0.276855	0.361625	0.253515	0.439017
Dragon	0.627996	0.625397	0.378959	0.624930	0.370748	0.578495
Electric	0.392420	0.327124	0.319836	0.581661	0.311204	0.512713

Fairy	0.463087	0.194444	0.348837	0.529412	0.428571	0.354839
Fighting	0.437200	0.476984	0.279734	0.426891	0.237415	0.537327
Fire	0.518736	0.456481	0.296318	0.561520	0.286508	0.463172
Flying	0.412752	0.222222	0.197674	0.358824	0.190476	0.541935
Ghost	0.471934	0.347475	0.293446	0.423797	0.240909	0.416129
Grass	0.461999	0.372973	0.261094	0.419078	0.263578	0.368439
Ground	0.530201	0.507310	0.331457	0.308359	0.226566	0.411885
Ice	0.508847	0.328283	0.215645	0.382353	0.233766	0.404692
Normal	0.469617	0.349850	0.197863	0.273132	0.179794	0.451613
Poison	0.454311	0.374786	0.258676	0.317195	0.225275	0.445658
Psychic	0.441540	0.413743	0.249204	0.518266	0.352632	0.527334
Rock	0.428955	0.445397	0.391362	0.348235	0.285170	0.366636
Steel	0.435937	0.462879	0.512051	0.343048	0.276407	0.326100
Water	0.515892	0.370755	0.288899	0.393452	0.269632	0.379550

1.5 Paso 4: Radar Chart Normalizado y Personalizado

Objetivo: Crear el gráfico radial definitivo con datos normalizados, eje fijo entre 0 y 1, y colores personalizados por tipo.

Instrucciones: 1. Extrae los valores normalizados de cada tipo usando `agrupado_norm.loc['Fire', :]` (igual que en el Paso 2 pero ahora de `agrupado_norm`). 2. Crea el `go.Figure()` y añade los tres `go.Scatterpolar`. Esta vez añade el parámetro `line_color` para asignar un color representativo a cada tipo (rojo para fuego, azul para agua, verde para planta). 3. En `fig.update_layout(...)`, configura el `radialaxis` con `range=[0, 1]` para que el eje esté fijo y sea comparable. 4. (Opcional): añade `opacity` a los trases para que los rellenos semitransparentes no se tapen entre sí. 5. Muestra el gráfico. ¡Ahora sí se pueden comparar los tres tipos claramente!

```
[7]: # Mismos tipos pero ya normalizados
fire2 = list(agrupado_norm.loc['Fire', :])
water2 = list(agrupado_norm.loc['Water', :])
grass2 = list(agrupado_norm.loc['Grass', :])

# Radar Chart con Plotly + colores intuitivos por tipo
# https://plotly.com/python/radar-chart/
fig = go.Figure()
fig.add_trace(go.Scatterpolar(
    r=fire2,
    theta=stats,
    fill='toself',
    name='Fire',
    line_color='red',      # rojo = fuego
    opacity=0.7           # transparencia para que se vean los solapes
))

fig.add_trace(go.Scatterpolar(
    r=water2,
```

```

        theta=stats,
        fill='toself',
        name='Water',
        line_color='blue',    # azul = agua
        opacity=0.7
    ))

fig.add_trace(go.Scatterpolar(
    r=grass2,
    theta=stats,
    fill='toself',
    name='Grass',
    line_color='green',    # verde = planta
    opacity=0.7
))

# Eje radial fijo en 0-1 para que la comparativa sea justa
fig.update_layout(
    title='Fire vs Water vs Grass (normalizado)',
    polar=dict(
        radialaxis=dict(visible=True, range=[0, 1])
    ),
    showlegend=True
)

fig.show()

```

Fire vs Water vs Grass (normalizado)



1.6 Reflexión Final del Alumno

Tómate un rato para reflexionar. * El **gráfico Radar Chart** es una herramienta muy potente para comparar perfiles multivariantes de forma visual. En este ejercicio hemos visto cómo el trío clásico de tipos (Fuego, Agua, Planta) tiene fortalezas y debilidades distintas: los Pokémon de tipo Fuego tienden a destacar en **Speed** y **Sp. Atk**, mientras que los de tipo Agua suelen ser más equilibrados. * Recuerda la lección más importante: **siempre normaliza los datos** antes de hacer un Radar Chart cuando las variables tienen escalas distintas. Sin normalización, el gráfico

puede ser visualmente engañoso o ilegible. * Como reto adicional: ¿podrías repetir el gráfico pero comparando **5 tipos** en lugar de 3? ¿O cambiar los tipos y ver qué tipos de Pokémon son más “redondos” (equilibrados en todas las estadísticas)?

Asegúrate de que cada celda tenga sus respectivos comentarios.

Metía Psychic y Dragon al trío Fire/Water/Grass. El más redondo me sale Water, las 6 stats le quedan parecidas sin tirar fuerte por ningún lado. Con más de 5 tipos ya no veo nada, se me solapan los rellenos.

```
[8]: # Reto: 5 tipos. Añado Psychic y Dragon al trío clásico
tipos_5 = ['Fire', 'Water', 'Grass', 'Psychic', 'Dragon']
colores_5 = ['red', 'blue', 'green', 'purple', 'orange']

fig = go.Figure()
for t, c in zip(tipos_5, colores_5):
    fig.add_trace(go.Scatterpolar(
        r=list(agrupado_norm.loc[t, :]),
        theta=stats,
        fill='toself',
        name=t,
        line_color=c,
        opacity=0.55          # bajo opacidad porque hay 5 capas que se solapan
    ))

fig.update_layout(
    title='Radar con 5 tipos (Fire/Water/Grass/Psychic/Dragon)',
    polar=dict(
        radialaxis=dict(visible=True, range=[0, 1])
    ),
    showlegend=True
)
fig.show()
```

Radar con 5 tipos (Fire/Water/Grass/Psychic/Dragon)

